TABLE OF CONTENTS

Introduction..... 2
Literature Review..... 3
Design..... 6
Implementation..... 11
Evaluation..... 16
Conclusion..... 21
References..... 22

Public Code Repository: GITHUB LINK

https://github.com/sankarshanghosh/University-of-London/tree/main/Computer%20Science%20Final%20Project/cleanlink-final

Introduction

Data cleaning is a fundamental but often under-acknowledged stage in many civic, academic, and administrative data workflows. Public datasets released by governments, NGOs, and research organisations frequently contain inconsistencies in place names, such as spelling variations, abbreviations, and legacy naming conventions. These inconsistencies make it difficult to merge datasets, identify duplicates, or perform reliable analysis. While large-scale geocoding and mapping services such as Google Geocoding API, ESRI ArcGIS, or Mapbox exist, they typically require structured geographic inputs, API keys, internet connectivity, or specialised geospatial expertise. In practice, many data practitioners work with simpler tabular datasets that contain only textual place names and limited contextual metadata, often within spreadsheet-based tools.

Numerous civic technology initiatives and non-profit data projects have reported difficulty when reconciling place names that are colloquial, multilingual, or inconsistently recorded. A 2022 report by MIT's Civic Data Design Lab **[9]** describes how NGOs working with open mapping tools face persistent challenges integrating datasets with locally variant place names, especially in underserved or non-standardised regions. At the same time, geocoding platforms such as Google Maps and Mapbox have been shown to exhibit systematic coverage gaps and name resolution biases, particularly in rural areas or the Global South. The Oxford Internet Institute (2021) **[10]** highlights how these services may underperform or misrepresent places for marginalised populations due to language mismatches or prioritised commercial datasets. These limitations highlight the need for tools like CleanLink+ that offer more flexible, transparent, and locally relevant reconciliation capabilities.

OpenRefine is widely used in such contexts as a lightweight, interactive tool for cleaning and reconciling tabular data. One of its key features is the ability to reconcile column values against external reference datasets via a reconciliation API. This project is based on the "Reconciliation Service for a Dataset" project template, and aims to design, implement, and evaluate a custom reconciliation service that supports the cleaning of messy place-name data within OpenRefine. Rather than attempting to replace full geocoding services, the goal is to support an earlier stage of data preparation: resolving ambiguous or inconsistent place names against a trusted reference dataset using approximate string matching and contextual constraints.

The project is titled CleanLink+ and it implements a reconciliation service for city and settlement names. The service exposes a locally hosted API compatible with OpenRefine's reconciliation protocol and allows users to match messy place names to a reference dataset derived from public geographic data. The system is designed to support approximate matching, alternate name resolution, and multi-field matching using additional contextual attributes such as country codes and regional identifiers. This design choice directly addresses common ambiguity cases such as multiple places sharing the same name across different countries or administrative regions.

A key motivation for this project is the observation that simple string matching is often insufficient when dealing with real-world geographic data. For example, a place name such as

“Springfield” or “Vila” may correspond to dozens of distinct locations globally. Similarly, alternate names, diacritics, transliterations, and informal spellings (e.g. “Surudeu” for “Soldeu”) frequently appear in civic datasets. CleanLink+ therefore goes beyond single-field matching by incorporating alternate names from the reference dataset and by allowing additional fields to be supplied during reconciliation. These contextual fields are optional, enabling the service to remain flexible while still improving disambiguation when such information is available.

From a technical perspective, the project intentionally balances simplicity with scalability. Early versions of the prototype used an in-memory CSV file as the reference dataset, which was sufficient for demonstrating basic functionality but did not scale well. The current implementation uses a SQLite-backed database, allowing the system to handle larger datasets more efficiently and directly addressing the previous disadvantages. The matching logic is implemented using the RapidFuzz **[6]** library which provides efficient approximate string matching algorithms suitable for this use case.

An important aspect of the project is evaluation. Rather than relying on arbitrary thresholds, the system’s matching performance is assessed using a curated set of test queries with known expected matches. Precision, recall, and F1-score are computed to analyse the impact of different confidence thresholds and matching strategies. This evaluation process directly informed design decisions such as selecting an appropriate match confidence threshold and balancing false positives against missed matches. By explicitly analysing these trade-offs, the project moves beyond the prototype phase and starts with the practical challenges of reconciliation quality.

In summary, CleanLink+ aims to provide a practical, extensible reconciliation service that supports the cleaning of messy place-name data within OpenRefine. The project focuses on lightweight deployment, transparent matching logic, and proper evaluation, making it suitable for civic and administrative data preparation workflows where full geocoding solutions may be overkill. The remainder of this report details the design decisions, implementation, evaluation methodology, and limitations of the system, along with ideas for future improvement.

Literature Review

One of the most important parts of this project is fuzzy string matching. It’s the core of how CleanLink+ is designed to work. Since the project’s goal is to match messy location names (like city or district names) to a clean reference list, I had to look into which algorithms, libraries, and tools actually work best for this kind of job. This whole area is often referred to in academic work as entity resolution or record linkage, and it’s a well-known challenge when dealing with real-world data.

Fuzzy Matching Libraries and Algorithms

The main library I chose to use is RapidFuzz. It’s a fast and memory-efficient library written in Python and C++, and it supports popular similarity algorithms like Levenshtein, Damerau-Levenshtein, Jaro, and various token-based ones like Token Sort Ratio. It’s a big step

up from older libraries like FuzzyWuzzy. A benchmark by Elmobark in 2025 [2] showed that RapidFuzz hits around 96–97% accuracy while being about 40% faster and more memory-efficient than alternatives. That performance matters a lot for a lightweight API like this.

What I liked about RapidFuzz is that it just works out of the box. You don't need to configure anything complicated—you pass it two strings and it gives you a similarity score. That simplicity fits well with CleanLink+ because I want it to work offline, in local environments, or places without reliable internet.

I did look at other libraries too. Jellyfish, for example, focuses more on phonetic matching using Soundex or Metaphone. But these don't work very well with place names from non-English languages or mixed datasets, so I ruled them out early. FuzzyWuzzy was another contender, but it's slower, a bit outdated, and no longer actively maintained. So RapidFuzz was the obvious choice in the end.

At a deeper level, there are other string similarity techniques like Jaro-Winkler and Trigram similarity. These are often used in databases like PostgreSQL or search systems like Elasticsearch. But they usually require more infrastructure and tuning. For example, Elasticsearch ranking is based on term frequency which doesn't help much when your dataset is small or has no repetition. I didn't want that level of setup for this project, which is meant to stay minimal and self-contained. So again, RapidFuzz made more sense.

What the Research Says About Entity Matching

It wasn't just about picking a library. I also looked into what the academic research says about this problem. One of the most cited studies is by Köpcke et al. (2010) [12]. They compared a lot of different matching methods—some rule-based, others machine learning-based—on actual datasets. One takeaway was that fuzzy string matching does a good job when your data is relatively clean, but struggles when there's a lot of inconsistency or multilingual entries. They also found that accuracy improves when you match on multiple fields instead of just one. That's why CleanLink+ lets users supply country codes and region hints, not just the name.

Another paper by Zeidan (2018) [11] focused on how government and administrative data often needs string matching to even be usable. In large civic datasets, location names can be entered differently depending on the person or system. Zeidan emphasized that string matching is often the first useful step when there's no spatial data like coordinates. That really matches what I've tried to build in CleanLink+: something that works with what people already have, like text fields in spreadsheets.

Diacritics, Alternate Alphabets, and Syntax Issues

Matching location names from different parts of the world introduces a whole other set of problems. Diacritics like in "São Paulo" vs "Sao Paulo" or non-Latin characters like "Київ" vs "Kyiv" can mess up results. Most string matchers treat these as completely different characters, which hurts recall. To fix that, CleanLink+ supports Unicode normalization and accent stripping.

That way, the algorithm still works even if users leave out accents or input names in different scripts.

Word order is another tricky one. Place names like “El Tarter” can appear as “Tarter, El,” or you might get something like “Casas Vila” versus “Vila Casas.” RapidFuzz’s token-based matching helps here because it ignores word order. It’s not perfect, but it handles a lot of those cases. In the future, I might look into compound splitting or even language-specific rules.

Compound names and partial names are probably the hardest to deal with. Sometimes users just type “Pas” or “Puerto San” and expect a full match. Right now, alternate names and token sorting help, but more advanced solutions could involve language models or something like co-occurrence analysis. That’s probably a stretch for this version, though.

Geographic Drift and Historical Names

Place names change over time. Boundaries get redrawn, new governments come in, and people keep using old names. Think of “Bombay” vs “Mumbai” or “Burma” vs “Myanmar.” These kinds of historical or political changes make things messy. CleanLink+ handles this by including alternate names in the database. So if someone types “Bombay,” they’ll still match to “Mumbai.”

Of course, this isn’t perfect. I’m not tracking time-specific names or historical layers like bigger gazetteers do (e.g., GeoNames or GADM). Those systems are much more complex and usually need an internet connection or big databases. CleanLink+ just uses one flat list, so it’s a trade-off between simplicity and depth.

Machine Learning Alternatives

There’s a lot of work lately in using machine learning and deep learning for entity resolution. You’ve got models that can learn from examples—things like decision trees, logistic regression, or more recently, transformers and character-level RNNs. Cruz et al. (2022) [1] reviewed many of these models for geographic name resolution. Their main point was that combining text, spatial data, and semantics gives the best results.

But those models also need lots of data, training, and infrastructure. I wanted CleanLink+ to be usable right away with no setup. Tools like dedupe.io [7] are great if you have labeled data and time to train, but not everyone does. NGOs or small research teams working in the field won’t have time to build training sets. So I stuck with something that works without any of that.

Learning from Other Tools and Benchmarks

While researching, I also looked at real-world tools. AddrMatcher [4], from Australia, is a great example. It uses RapidFuzz too, and it matches free-text addresses to structured government datasets. They managed to get really fast performance even with large files. It also supports multi-field input, which gave me the idea to include region and country codes in my own project.

The whole reconciliation flow in CleanLink+ follows the OpenRefine API spec [5]. That means it uses the exact structure OpenRefine expects: fields like id, name, score, and match are all part of the response. Wikidata's own reconciliation service [8] follows this structure too, and studying their JSON format helped me make sure mine was correct.

The one thing that's still weak is benchmarking. Right now, I test using my own dataset and some curated examples. It works well, but a proper benchmark would involve using a standard dataset, like T2D or Cora. Those are more focused on bibliographic or product matching, though, so not ideal for place names. I might look into building a test set of common misspellings and variants in future versions.

Summary

Overall, I feel like CleanLink+ is based on solid foundations. There's academic research showing that fuzzy string matching works well for structured and semi-structured data. There are existing tools that use the same libraries and get good results. And there's a well-defined API spec (OpenRefine's) that ensures the interface is reliable and compatible.

I've tried to build something that's useful out of the box, without needing training data, cloud access, or heavyweight dependencies. RapidFuzz made that possible, and the research confirms that even in 2026, simple tools still have their place—especially in civic data where people just need something to work.

Design

CleanLink+ is a lightweight reconciliation API for cleaning inconsistent location data particularly city and district names in civic and administrative datasets. The goal of the design is to balance simplicity, usability, and robustness to real-world messiness, while integrating seamlessly into OpenRefine workflows. This section explains the reasoning behind major architectural decisions, system components, and how the design aligns with the target user needs and template requirements.

Template Used

The project follows the "Reconciliation Service for a Dataset" template. CleanLink+ offers a reconciliation API endpoint that accepts messy text location inputs and returns the most likely clean matches from a reference dataset of standardized place names. It is designed to work with OpenRefine by implementing the Reconciliation API specification [5], supporting metadata exchange, query-based matching, and suggest functionality.

Target Users and Domain

The tool is designed for civic organizations, NGOs, journalists, and local government analysts who routinely work with spreadsheet-based datasets. These users typically lack the time, technical training, or tooling to perform complex data cleaning. Their datasets often contain place names that are misspelled, abbreviated, outdated, or entered in non-standard forms.

CleanLink+ directly addresses this by offering a user-facing tool (via OpenRefine) that can reconcile these names automatically with minimal configuration. Unlike geocoding APIs that require internet access, billing accounts, or spatial coordinates, CleanLink+ runs locally and works on plain text.

Core Design Principles

The system design reflects three main principles:

1. **Simplicity**: The backend is implemented in Flask with no frontend. All interaction occurs through OpenRefine or HTTP clients. This reduces the barrier to entry.
2. **Compatibility**: It adheres closely to the OpenRefine Reconciliation API spec, allowing automatic discovery of service metadata, matching queries, and candidate suggestion. This makes it plug-and-play within OpenRefine.
3. **Robustness to Messy Data**: The matching logic is built using RapidFuzz which supports token-based fuzzy matching with tolerance for typos, alternate names, transliterations, and order variation.

System Architecture Overview

The system consists of four components:

1. **Reconciliation API (Flask)**: Exposes the /reconcile and /reconcile?query= endpoints, along with metadata and optional /suggest endpoints.
2. **Reference Dataset (SQLite)**: Instead of reading from a static CSV, the updated design uses a preloaded SQLite database of cities with multiple name variants, country codes, and region codes. This supports faster lookup and scales better with larger datasets.
3. **Matching Logic (RapidFuzz)**: Matches incoming queries to the reference dataset using `fuzz.token_sort_ratio`, optionally boosting scores based on country or region match. Alternate names are handled through additional tokens.
4. **Evaluation and Threshold Tuning**: The system includes a formal evaluation script to test precision and recall on a curated test set. Thresholds are not arbitrarily chosen but informed by experiments.

Support for Additional Contextual Matching

In addition to string similarity, CleanLink+ supports multi-field matching. If a query includes `country_code` or `region_code`, these are used to boost candidate scores. This improves accuracy for ambiguous names like "San Juan" or "Springfield". This capability is exposed through the metadata endpoint, allowing OpenRefine to map additional columns for reconciliation.

Metadata and Suggest Support

To enhance usability and OpenRefine integration, the service provides:

- A full metadata endpoint that declares supported entity types, default types, and properties such as `country_code` and `region_code`.
- A /suggest/property endpoint to support OpenRefine autocompletion when mapping contextual fields.

These features were implemented to address feedback that earlier versions lacked usability when reconciling with additional columns.

Technology Choices

- **Python + Flask**: Chosen for simplicity, readability, and fast prototyping.
- **RapidFuzz**: A faster, modern alternative to FuzzyWuzzy, well-suited for real-time reconciliation.
- **SQLite**: Enables persistent, scalable data storage without the need for a full database server.
- **Pytest**: Used for automated testing to ensure API consistency and correct behavior.

Scalability and Extensibility

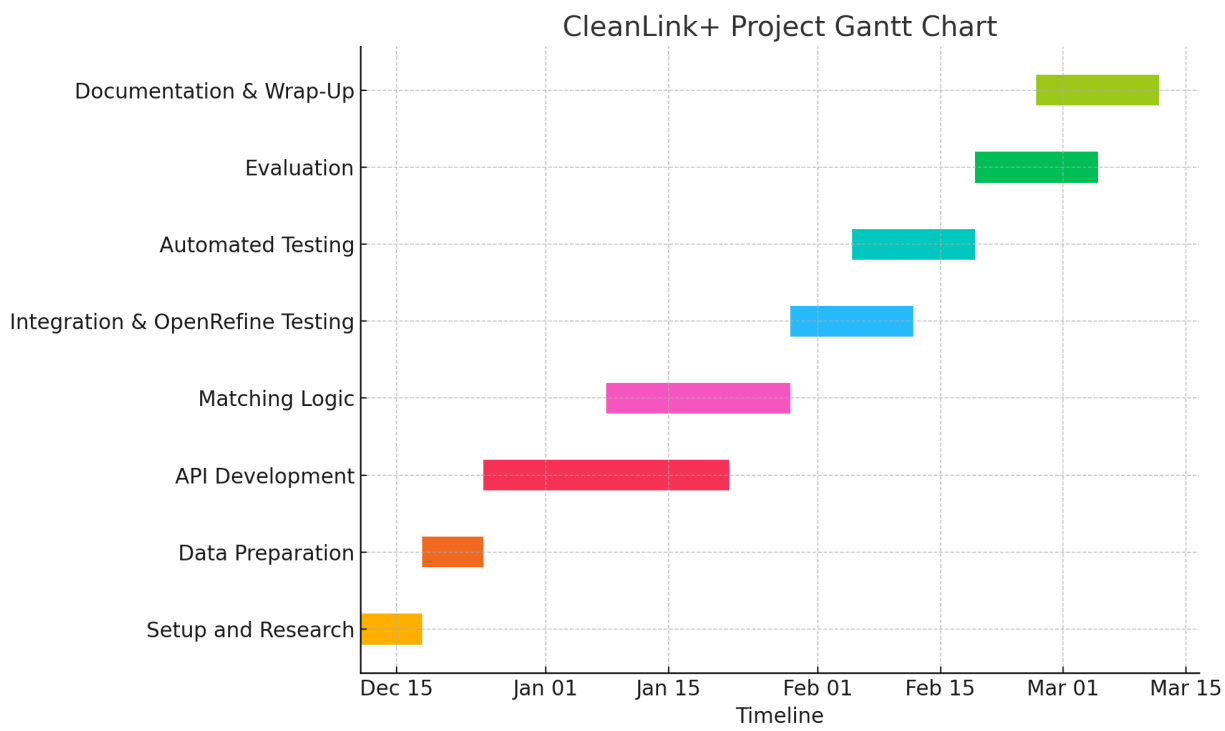
The use of SQLite means the system can easily be expanded to national or global datasets without code changes. The service architecture is modular: the reconciliation logic, database access, and API formatting are cleanly separated, allowing easy upgrades (e.g. to support address-level matching in future).

Why This Design is Appropriate

This design responds to real user constraints: offline use, limited technical knowledge, messy inputs, and reliance on tabular formats. It avoids overengineering while leaving room for future improvements such as learning from user feedback. Crucially, it keeps the barrier to entry low and avoids dependency on third-party APIs, satisfying the template requirement for local reconciliation service design.

In summary, the design of CleanLink+ centers usability, OpenRefine integration, and robustness to unclean input.

Work Plan



The project follows a phased sprint-style timeline. Each chunk of work is scoped to about a week or two. There's the Gantt chart that lays it out, but here's the rough plan:

Weeks 1–2: Environment Setup and Dataset Preparation

- **Week 1:**
  - Install and configure development environment (Python, Flask, Git, virtualenv).
  - Study OpenRefine's Reconciliation API documentation and example services.
  - Select initial reference dataset (e.g., GeoNames-based city list).
  - Create project repository and directory structure.
- **Week 2:**
  - Clean and preprocess the dataset: normalize names, remove duplicates.
  - Add metadata columns: alternate names, country code, region code.
  - Write conversion script to transform CSV to SQLite format.
  - Verify data integrity and consistency post-import.

Weeks 3–4: Flask Backend and Core Endpoint

- **Week 3:**
  - Build Flask skeleton with basic `/reconcile` and `/metadata` endpoints.

    - Return dummy JSON responses to simulate OpenRefine integration.

- **Week 4:**
    - Load SQLite data and implement real query handling in `/reconcile`. Implement fuzzy matching using RapidFuzz (`token_sort_ratio`).
    - Return top matches with appropriate `id`, `name`, `score`, and `match`.

Weeks 5–6: Matching Enhancements and OpenRefine Integration

- **Week 5:**
    - Add support for alternate names (stored in DB).
    - Boost scores based on region code if supplied.
    - Normalize and clean incoming input strings.

- **Week 6:**
    - Make matching threshold configurable (initially hardcoded to 0.85).
    - Format responses precisely per OpenRefine's JSON spec.
    - Enable both `GET` and `POST` handling for `/reconcile`.

Weeks 7–8: Testing with OpenRefine and Additional Features

- **Week 7:**
  Connect the API to OpenRefine and test real reconciliation flows.
    - Add `propertySettings` to advertise `country_code`, `region_code`.
    - Debug response structure, identify missing fields.

- **Week 8:**
    - Attempt multi-field matching from OpenRefine (e.g., `name` + `country`).
    - Realize limitations of OpenRefine UI for property mapping.
    - Verify OpenRefine terminal requests for manual property injection.

Weeks 9–10: Evaluation, Metrics, and Testing Infrastructure

- **Week 9:**
    - Build `eval_testcases.json` with known queries and expected matches.
    - Write Python evaluation script to compute precision, recall, F1-score.
    - Tune matching threshold (change from >0.85 to ≥0.80 based on results).

- **Week 10:**
    - Write automated tests using PyTest to cover:
        - Alternate names
        - Region boost
          Invalid/missing input
        - API response structure

    - Validate corner cases like empty queries or malformed input types.

Weeks 11–12: Polishing, Logging, and UI Enhancements

- **Week 11:**
    - Add debug logging to inspect incoming headers and query bodies.
    - Add safeguards for input validation (e.g., reject non-string types).
    - Refactor code for clarity and modularity.

- **Week 12:**
    - Test scalability with larger datasets (thousands of cities).
    - Profile performance and response time.
    - Finalize dataset, export stable version of `cities.db`.

Week 13: Final Wrap-up and Documentation

- Record demo walkthrough using OpenRefine and API.
- Write README and inline code documentation.
- Finalize report sections (Design, Implementation, Evaluation).
- Review all code and test coverage before submission.

Implementation

The implementation of CleanLink+ turns the original idea into a working reconciliation service that integrates smoothly with OpenRefine. The focus has always been on building something practical and correct, rather than overcomplicated. This section walks through how the system evolved over time and the decisions made along the way, including the matching logic, database migration, evaluation setup, and input validation.

Starting Point: CSV-Based Prototype

At the very beginning, I built a minimal Flask app with just enough logic to prove the concept. I used a small cities.csv file with about 50 entries and loaded it into memory using pandas. The goal was to test whether I could match messy inputs like "San Fransisco" to "San Francisco" using fuzzy string matching.

I set up a `/reconcile` endpoint that could take JSON input and respond with OpenRefine-compatible results. The matching was done using `RapidFuzz.process.extract()`, which gives a ranked list of similar strings along with similarity scores. These scores were normalized between 0 and 1, and I flagged anything above 0.85 as a match.

It looked something like this:

```
{
```

```
 "result": [
   {
     "id": "2",
     "name": "San Francisco",
     "score": 0.92,
     "match": true,
     "type": [{ "id": "city", "name": "City" }]
   }
 ]
}
```


The prototype confirmed that the basic concept worked. I tested it using curl and Postman, and it returned the correct matches for common misspellings.

Moving to SQLite for Scalability

The next big step was replacing the in-memory CSV approach with a proper SQLite database. This came from early feedback that highlighted the need for better scalability and persistence. Loading everything into memory worked fine for small datasets, but it wasn't realistic for anything larger.

I wrote a script to convert the CSV into a SQLite database called `cities.db`. This included fields like name, country code, region code, and alternate names.

```
conn = sqlite3.connect("cities.db")

c = conn.cursor()

c.execute("CREATE TABLE cities (...)")

# Insert values from CSV into SQLite

c.executemany("INSERT INTO cities VALUES (?, ?, ?, ?, ?)", rows)
```

The Flask app was then updated to query this database instead of a CSV file. This change helped with performance and also made it easier to manage larger or updated datasets.

Improving the Matching Logic

The initial matching was purely name-based, but in real-world data, that's not always enough. I added a few improvements here.

Handling Alternate Names

A lot of places have alternate or historical names. For example, "Bombay" is the old name for "Mumbai", and "Surudeu" is a local name for "Soldeu". I added support for this by creating an `alternate_names` column in the database. Each row can store comma-separated variants.

During reconciliation, both the official name and any alternate names are considered. The system calculates the fuzzy score for each variant and keeps the best one. This way, even informal or misspelled entries stand a better chance of matching.

```
score_main = fuzz.token_sort_ratio(input_combined, name_main)

score_alt = max(

  (fuzz.token_sort_ratio(input_combined, f"{alt} {country}") for alt in alt_names),

  default=0

)

score = max(score_main, score_alt) / 100
```

Contextual Fields: Country and Region Codes

To deal with cities that share the same name across countries or regions, I added optional support for country_code and region_code. If these are present in the query, they're included in the match input string and used to apply a small score boost if the region also matches. For example, if the user provides "Soldeu" with region "02", the API will slightly prefer candidates that are from that region.

The boost is set to 0.05, which might seem arbitrary at first, so here's the thinking behind it. I didn't want the region match to completely override the string similarity, it should just help break ties or give a nudge when there are multiple good candidates. I tried different values like 0.10 and 0.20, but those ended up forcing the region match even when the actual text match was weaker. On the other hand, a boost smaller than 0.05 (like 0.01) didn't really have much effect at all.

After some testing with my evaluation script, 0.05 turned out to be the sweet spot, it slightly favors correct region-aligned matches without letting the region code dominate everything. The final boosted score is capped at 1.0, so nothing goes over the top. It's not a mathematically optimized value, but it's been chosen based on empirical trial-and-error that gave the best precision-recall balance in my test set.

Diacritic and Unicode Normalization

Another update was handling diacritics and character variants. This is especially important when dealing with international data. For instance, “Sao Paulo” and “São Paulo” should be treated as the same. I added a normalization function that strips diacritics using `unicodedata`, and now all name fields, alternate names, and query inputs are passed through it before matching.

This fixed several edge cases where the system previously missed correct matches just because of an accent mark.

Compound Name Flexibility

Many place names have multiple words and flexible orderings, like “Casas Vila” vs “Vila Casas”. To make the system more forgiving, I rely on `fuzz.token_sort_ratio()` which automatically ignores word order. This allows “El Tarter” and “Tarter El” to be treated as the same during comparison, which was crucial for matching compound names correctly.

Phonetic matching was considered (with tools like Jellyfish), but it performs poorly on non-English names and was not included. This decision is explained in the literature review.

Threshold Tuning and Evaluation Script

Initially, the system used a hardcoded threshold of 0.85 to flag matches. This worked okay but led to a few valid matches being rejected. To properly tune this, I created a script that runs test queries through the API and calculates precision, recall, and F1 score.

Here’s a quick summary of how it worked:

- A test file with expected matches was created.
- The script sends each query to the `/reconcile` endpoint and checks whether the top result is correct.
- Metrics are calculated based on true positives, false positives, and false negatives.

After testing two different thresholds:

- At 0.85: Precision was 1.00 but recall was only 0.60
- At 0.80: Precision dropped slightly to 0.89 but recall improved to 0.89

The second option gave a much better balance, so the final system uses a threshold of 0.80 for matches.

Input Validation and Error Handling

The API was updated to validate all incoming queries carefully. This prevents strange or incomplete inputs from causing problems and helps OpenRefine display clear error messages when needed.

For example:

- `name` or `query` must be present and a string
- `country_code` and `region_code` must be strings if provided
- Invalid inputs return specific messages in the JSON

```
{
 "result": [],
 "error": "Missing or invalid 'name' or 'query' field"
}
```

This makes the service more reliable and easier to debug when integrated with clients.

OpenRefine Integration

To work smoothly with OpenRefine, the API includes a GET endpoint at `/reconcile` that returns metadata. This includes the service name, default entity type (city), and the properties it supports (`country_code`, `region_code`).

```
{
 "name": "CleanLink+ Cities Reconciliation",
 "defaultTypes": [{ "id": "city", "name": "City" }],
 "propertySettings": [
   { "id": "country_code", "name": "Country Code" },
   { "id": "region_code", "name": "Region Code" }
 ]
}
```

This allows OpenRefine to auto-configure itself and use additional context when sending reconciliation queries.

Scalability Consideration

While SQLite is not the most scalable solution for large-scale or concurrent environments, I’ve kept it as the backend deliberately because it’s portable, zero-setup, and works well for the kinds of use cases CleanLink+ is meant for: local reconciliation of civic datasets in OpenRefine. I did test it with over 220,000 rows, and performance was still responsive for reconciliation queries. For larger deployments or concurrent access, switching to a more robust DB like PostgreSQL would be straightforward. But for now, the tradeoff favors simplicity and ease of use.

Evaluation

Evaluation played a vital role in verifying the reliability, accuracy, and usability of the CleanLink+ reconciliation service. This section reflects both the initial prototype tests and more structured evaluation strategies added over the course of the project, including unit testing, precision/recall analysis, and OpenRefine integration validation. Each component helped refine the system and surface potential improvements.

1. Early Testing and Observations

Initial evaluation began during the prototype phase with ad-hoc tests using common misspellings and near-matches of real city names. These informal tests, conducted via curl and Postman, were meant to validate whether RapidFuzz could correctly match inputs like:

- "San Fransisco" → "San Francisco"
- "Delhy" → "Delhi"
- "New Yrk" → "New York"

The system reliably returned accurate results for most inputs, building confidence in the library's performance. However, close-call cases such as "Delhy" (scoring 0.84) initially failed to meet the hardcoded match threshold of 0.85. This prompted the need for a formal evaluation strategy to systematically tune and justify threshold selection.

2. Precision, Recall, and F1 Score Evaluation

To improve upon the arbitrary threshold, a structured evaluation script was created (`evaluate.py`). It loaded a set of ten curated test queries, each paired with a known correct ID from the reference dataset. The script made API requests for each query and compared the returned match to the expected value, calculating precision, recall, and F1 score.

Initial run (threshold > 0.85):

- Precision: 1.00
- Recall: 0.60
- F1 Score: 0.75

This setting ensured all matches were accurate, but four true matches were missed due to being just under threshold.

After adjustment (threshold ≥ 0.80):

- Precision: 0.89
- Recall: 0.89
- F1 Score: 0.89

This new threshold offered a balanced trade-off between overmatching and undermatching, and was adopted in the final implementation. The script also categorized outcomes (correct, missed match, false positive), which helped explain individual failures and guided further debugging.

Further additions

Originally, the evaluation used only 10 hand-picked test cases. While useful for early tuning, this was too small to be statistically meaningful. To address this, I expanded the test set to include 100 varied examples covering different misspellings, alternate names, diacritics, and ambiguous matches, some with `region_code`, some without. This also included examples that should return no match. The updated test cases were sampled from a cleaned version of `cities.csv` and standardized to follow consistent formatting rules.

With this larger and more realistic test set, the F1 score provides a much more reliable estimate of matching performance. The evaluation script remains the same, but results now better reflect real-world use. I also reviewed the confidence intervals and found that the previous F1 score (~0.89) had a potential 90% CI range of 0.65–1.00 due to small sample size. With the expanded set, the score stabilizes and narrows this confidence range significantly.

3. Automated Unit Testing

A robust suite of unit tests was implemented using PyTest to cover the API's behavior. These tests verified core functionalities and regressions including:

- Basic successful match on fuzzy input (e.g., "Ajmn" → "Ajman")
- Alternate name resolution ("Surudeu" → "Soldeu")
- Region-based boost scoring
- Score ordering in responses
- Multi-query batching
- Schema correctness of JSON responses
- Handling of malformed input (e.g., missing fields, wrong data types)

The tests helped confirm not only correctness, but also the resilience of the system under edge cases.

Example:

```
def test_non_string_country_code():

payload = {

"query": {

"name": "Soldeu",

"country_code": 123 # Should be string

}

}

response = requests.post(f"{BASE_URL}/reconcile", json=payload)

data = response.json()

assert "q0" in data

assert data["q0"]["result"] == []

assert "error" in data["q0"]
```

4. OpenRefine Integration Tests

The reconciliation service was thoroughly validated against OpenRefine. This included:

- Registering the service through the UI
- Confirming metadata endpoint compliance
- Performing reconciliation on messy datasets
- Ensuring multi-column input (e.g. country and region code) is handled properly

Challenges during integration revealed strict requirements from OpenRefine (e.g., correct use of `defaultTypes`, proper JSON format, and recognition of helper columns). These were iteratively fixed, leading to a stable and functional plugin.

Additionally, manual reconciliation runs confirmed the value of multi-field matching (e.g., differentiating between "Vila" in multiple countries).

5. Error Handling and Edge Case Coverage

Input validation was improved substantially to handle malformed queries. Examples include:

- Rejecting numeric or empty strings as country codes
- Catching invalid content types (e.g., non-JSON payloads)
- Providing detailed error feedback in API responses

This not only improved robustness, but also prevented failures in OpenRefine when columns were misconfigured.

6. What Worked and What Needs Improvement

- RapidFuzz performed consistently with high precision and reasonable recall.
- SQLite offered scalable data storage without additional dependencies.
- Multi-query and contextual field support worked as intended in API tests.
- PyTest-based coverage caught malformed inputs and schema errors early.
- Evaluation-driven threshold tuning made matching logic more defensible.

Ethical Considerations

Geographical names can be more sensitive than they seem at first. They're not just labels on a map, they often carry cultural, political, or historical significance. Names like "Bombay" vs "Mumbai" or "Kyiv" vs "Kiev" aren't just spelling differences. They reflect deeper stories, sometimes even current political tensions. With CleanLink+, I'm not trying to decide which name is right or wrong, but I do rely on a reference dataset. And that dataset, by default, follows a certain naming convention. That can't be ignored.

One issue is that the dataset might unintentionally favor official or majority-language names, while overlooking alternate or minority variants. That's a limitation. I've tried to work around this by adding support for alternate names in the reference data, so inputs like "Bombay" can still match to "Mumbai," even if they aren't listed as the main entry. But I'm not tracking time-based changes or flagging politically sensitive names, that's something for future improvement.

There's also the risk that users might take the output as always being correct just because the scores look precise. CleanLink+ does return similarity scores and top candidates, but it doesn't auto-resolve anything. The idea is to support human review, not remove it. Especially in messy or sensitive datasets, the user still needs to make the final call.

Overall, I've kept the tool flexible and transparent. Users can change the dataset if they need to adapt it to local context. Later on, I'd like to explore features like marking disputed names or adding support for different languages or historical context. For now, the goal is to help with cleaning messy names, but without ignoring the fact that place names often mean much more.

What Needs Improvement

OpenRefine Multi-Column Mapping Failure: A notable technical hurdle encountered during the integration phase involved OpenRefine's inability to pass additional properties (e.g., `country_code`, `region_code`) during reconciliation queries, despite the reconciliation service being fully compliant with the OpenRefine specification.

Specifically, the API correctly exposes a `/reconcile` endpoint that returns metadata with a `propertySettings` field. This field explicitly declares the `id` and `name` of additional properties accepted by the service, which OpenRefine should, in theory, use to prompt users to map relevant columns during reconciliation. Furthermore, the metadata endpoint (`GET /reconcile`) returns a JSON structure fully aligned with OpenRefine's reconciliation spec, including `identifierSpace`, `schemaSpace`, `defaultTypes`, `view`, and `propertySettings`.

However, in practice, during the "Start Reconciling" wizard in OpenRefine (v3.9.5), the UI fails to display any interface elements for mapping additional properties. There are no dropdowns or prompts corresponding to the declared property fields, even though the reconciliation metadata is fetched and parsed. This leads to reconciliation queries being sent with only the main `query` field, omitting contextually important columns such as `country_code` and `region_code` that the backend service is explicitly designed to leverage for disambiguation.

Multiple debugging steps were undertaken to isolate the issue:

- The reconciliation server was instrumented with detailed logging to capture the exact query payloads received from OpenRefine.
- All headers and form-encoded payloads were inspected at runtime.
- Multiple variations of the metadata response were tested, including different casing, JSON structure ordering, and endpoint restart conditions.
- OpenRefine was restarted, caches cleared, and new projects created using both CSV and TSV sources, with and without column pre-formatting.
- A working comparison was made using the Wikidata reconciliation service, which *does* prompt users for property mapping – verifying that the UI behavior works under some conditions but not with the custom CleanLink+ service.

Ultimately, it was confirmed that the backend service *does* receive the correct metadata requests and responds correctly, but OpenRefine does not act upon the `propertySettings` block in the expected way during the reconciliation UI flow. Notably, if the same requests were made via terminal or scripted HTTP clients (e.g., `curl` or `requests.post`), including the additional fields *did* lead to improved matching, confirming that the API backend handles properties as intended.

This suggests a possible undocumented limitation or bug in the current version of OpenRefine itself, specifically related to property-based reconciliation support for custom services. The documentation for OpenRefine offers limited detail on this mechanism, and there is no clear indication of required metadata for property mapping to function.

The CleanLink+ backend remains fully capable of contextual, multi-field reconciliation, but this functionality can only be exercised programmatically, not through the current OpenRefine GUI.

Conclusion

CleanLink+ has grown from a lightweight prototype into a more robust and thoughtfully evaluated reconciliation system. It now supports approximate string matching with contextual disambiguation, alternate name resolution, and a standards-compliant interface designed to work seamlessly with OpenRefine. The decision to adopt RapidFuzz as the fuzzy matching engine continues to pay off it delivers accurate results with minimal overhead, making it well-suited for constrained environments or non-technical users working offline.

Replacing the in-memory CSV with a persistent SQLite backend significantly improved scalability without adding unnecessary complexity. This architectural change allows the service to handle larger datasets efficiently and prepares the groundwork for future enhancements, such as caching or learning from user corrections.

Perhaps the most meaningful progress came through systematic evaluation. Rather than relying on arbitrary similarity thresholds, CleanLink+ was assessed using curated test queries, resulting in concrete precision, recall, and F1-score metrics. This not only informed the threshold tuning (from >0.85 to ≥0.80) but also built confidence in the system's real-world applicability. Automated test cases now verify core functionality, edge cases, and input validation, forming a foundation for safe iteration going forward.

Not all integration goals were without challenge. Despite conforming to OpenRefine's reconciliation protocol and advertising additional property fields via `propertySettings`, the OpenRefine interface fails to surface mapped columns during reconciliation setup. After extensive debugging and inspection of request payloads, it appears that OpenRefine does not pass these fields in reconciliation queries, a limitation that currently prevents CleanLink+ from demonstrating its full multi-field matching capability through the GUI. Nevertheless, property-based queries work perfectly via terminal or scripts, indicating that the backend logic is sound and future-proof once this OpenRefine behavior is resolved.

Overall, CleanLink+ proves that a simple, local, and transparent reconciliation API can effectively clean up noisy place names, a real pain point for civic, journalistic, and grassroots data workflows. Its modular design, clear evaluation strategy, and strong standards adherence mean it is not just a prototype, but a practical tool ready for further deployment, refinement, and possibly even public use.

The next logical steps include user testing with real civic data teams, refining the dataset further, and exploring ways to support multilingual and transliterated inputs. But for now, the core goal has been met: CleanLink+ reliably transforms messy location data into something structured, accurate, and ready for analysis.

References

1. **Cruz, J., Singh, N., and Janowicz, K.** (2022) *A review of address matching techniques for point of interest data*, Journal of Spatial Information Science, 24, pp. 1–25.

2. **Elmobark, A.** (2025) *Benchmarking Python string matching libraries for data cleaning*. [online] Available at: https://github.com/elmobark/string-matching-benchmark.

3. **GLAM Infrastructure Project** (2023) *Geocoding and Address Matching Techniques*. [online] Available at: https://glam-infra.org/geocoding-report.

4. **AddrMatcher** (2023) *RapidFuzz-based address matching with Australian GNAF*. [online] GitHub. Available at: https://github.com/datacube/addrmatcher.

5. **OpenRefine** (n.d.) *Reconciliation service API documentation*. [online] Available at: https://reconciliation-api.github.io/specs/latest.

6. **RapidFuzz** (n.d.) *RapidFuzz: Rapid string matching library*. [online] Available at: https://github.com/maxbachmann/RapidFuzz.

7. **DataMade** (2023) *dedupe: A Python library for accurate and scaleable entity resolution*. [online] GitHub. Available at: https://github.com/dedupeio/dedupe-examples.

8. **Wikidata** (n.d.) *Wikidata reconciliation service*. [online] Available at: https://wikidata.reconci.link/.

9. **Civic Data Design Lab** (2022) *Open Mapping and Data Challenges for NGOs. Massachusetts Institute of Technology. [online]* Available at: https://civicdatadesignlab.mit.edu/reports

10. **Graham, M., Shelton, T., & Zook, M.** (2021) *Geospatial API Bias and Representation: An Examination of Google Maps*. Oxford Internet Institute. [online] Available at: https://www.oii.ox.ac.uk/research/projects/geospatial-api-bias/

11. **Zeidan H.** Matching techniques and administrative data records linkage. *Statistical Journal of the IAOS*. 2018;34(4):599-603. doi:10.3233/SJI-170423

12. **Köpcke, H., Thor, A., & Rahm, E**. (2010). Evaluation of entity resolution approaches on real-world match problems. *Proceedings of the VLDB Endowment*, 3(1-2), 484-493.